SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction:

This section "BudgetBrain" introduces the strategic solution of the identified problem statement highlighted in Product Requirement Document.

1.1. Purpose:

The system analysis document highlights the technical scope and design-related decisions behind the BudgetBrain development.

So, the key element of system that has been covered are as followed:

1. ***Architecture**:*
   It is a simplified client-server architecture. The frontend (Streamlit) handles user input for 5 categories: Monthly Spending Template, Monthly Allowance, Passive Income, Investment, Future Goal. The only external service is Z.AI's GLM API. No database in MVP.
2. ***Data Flows:***
   How financial data (spending template, allowance, passive income, investments, goal) flows from user input, gets constructed into a prompt, sent to GLM API, parsed into 5 outputs, and displayed to user.
3. ***Model Process:***
   End-to-end workflow: Enter 5 inputs → GLM analyzes → Returns 5 outputs (Recommendation, Reasoning, Expected Impact, Confidence, Reminder) → User accepts/rejects.
4. ***Role of Reference:***
   This document is a reference for developers and testers for the overall high-level architecture.

1.2. Background:

The team identified that personal finance tools don't connect spending with multiple income sources (allowance + passive income) AND future goals. Users need accountability through reminders.

1. *Previous Version:*
   N/A - New project for UMHackathon 2026.
2. *Changes in Major Architectural Components:*
   N/A
3. *New capabilities introduced:*
   - *5-input analysis (Spending + Allowance + Passive Income + Investment + Goal)*
   - *5-output decision intelligence (including Reminder)*

© UMHackathon 2026

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
End Users (Students/Professionals)	*Enter spending template, allowance, passive income, investments, goal; receive GLM recommendations*	*Simple input process, clear 5-output decision, actionable reminder*
Judges (UMHackathon)	*Evaluate system against criteria*	*GLM in critical path, 5 distinct outputs, quantifiable impact, reminder feature*
Development Team	*Build the platform and maintain it*	*Clear API usage contracts, codebase is modular and well-documented architecture*
QA Team	*Validate system*	*Defined scope, request/response formats (5 sections), edge cases*

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	*Web Application (Streamlit)*
Architecture	*Client-Server with external GLM API*

BudgetBrain is structured as a client-server based system with 1 client type: User Browser accessing Streamlit app. The system calls GLM API as the decision intelligence service. No database - session state stored in memory.

2.1.2 LLM as Service Layer

The architecture of your product needs to show how GLM is integrated as a service layer- not limited to a generic "AI Module". The following Dependency diagram includes:

2.1.3 Dependency Diagram

a. *How the prompts are being constructed and sent to the GLM API:*

Step 1: Collect all 5 inputs
- Monthly spending template (categories + amounts)
- Monthly allowance
- Passive income
- Investment details
- Future goal (description + timeline)

Step 2: Construct prompt with few-shot examples
- Format all inputs clearly
- Add 2 example good responses with 5 sections
- Total: ~1200-1500 tokens

Step 3: Send HTTP POST to GLM API
- Endpoint: [TBD - waiting for API]
- Headers: Authorization + Content-Type
- Body: {model, messages, temperature=0.3}
- Timeout: 10 seconds

© UMHackathon 2026

b. *What goes into the context window:*
   - *Monthly spending template (categories and amounts)*
   - *Monthly allowance amount*
   - *Passive income amount and source*
   - *Investment details (type, amount, return if known)*
   - *Future goal (description and timeline)*
   - *2 few-shot examples of good 5-output responses*
   - *Format instructions (5 bolded sections)*

c. *How is your system receiving, parsing the responses and passing to the next system component:*

Step 1: Extract content
response["choices"][0]["message"]["content"]

Step 2: Validate ALL 5 sections exist
- Recommendation:
- Reasoning:
- Expected impact:
- Confidence:
- Reminder:

If valid : *Display 5 outputs to user*

If invalid/missing :
- Retry once with temperature = 0.1
- Graceful error message

d. *Where the limitation of token are enforced or input are chunked before it actually reaches to GLM:*
- Maximum input: *1500 tokens*
- ***Enforcement point:*** *Before prompt construction*
- ***If exceeded:*** *Summarize spending template (keep top 4 categories)*

e. *Major API calls:*

Caller	Callee	Purpose
Streamlit Frontend	*Python Backend*	*User input handling*
Python Backend	*GLM API*	*Get 5-output decision*
Python Backend	*Session State*	*Store accept/reject*

© UMHackathon 2026

f. How component interact:
- No database in MVP
- GLM API is the only external service
- All logic runs in Streamlit process

g. Distributed components:
- System is NOT distributed (runs on single machine)
- GLM API is external distributed service

2.2.2. Sequence Diagram

Step	Actor	Action	System Response
1	User	Enters monthly spending template (Food: RM400, Transport: RM150, Entertainment: RM200)	System stores in session state
2	User	Enters monthly allowance (RM800)	System stores in session state
3	User	Enters passive income (RM200 from tutoring)	System stores in session state
4	User	Enters investments (RM500 savings account)	System stores in session state
5	User	Enters future goal ("Save RM2000 for laptop in 5 months")	System stores in session state
6	User	Clicks "Ask GLM" button	System constructs prompt with all 5 inputs
7	System	Sends HTTP POST request to GLM API	Shows loading spinner
8	GLM API	Returns JSON with 5 outputs	System parses response
9	System	Validates all 5 sections exist	If missing, retry with temperature=0.1
10	System	Displays Recommendation, Reasoning, Expected Impact, Confidence, Reminder	User sees decision
11	User	Clicks Accept or Reject	System tracks decision in session state

© UMHackathon 2026

2.2. Technological Stack

2.2.1. Frontend: Streamlit (Python) - Fast prototyping

2.2.2. Backend: Python (same as frontend) - GLM integration module

2.2.3. Database: None (MVP) - Session state only

2.2.4. Cloud/Deployment: Localhost (preliminary round)

2.2.5. External API: Z.AI GLM API

2.2.6. Additional Libraries: requests, streamlit

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored

2.3.1.1. ***Data Flow Diagram (DFD)***

a. External Entities: User

b. Processes:

1. Collect 5 Inputs (Spending, Allowance, Passive, Investment, Goal)

2. Construct Prompt

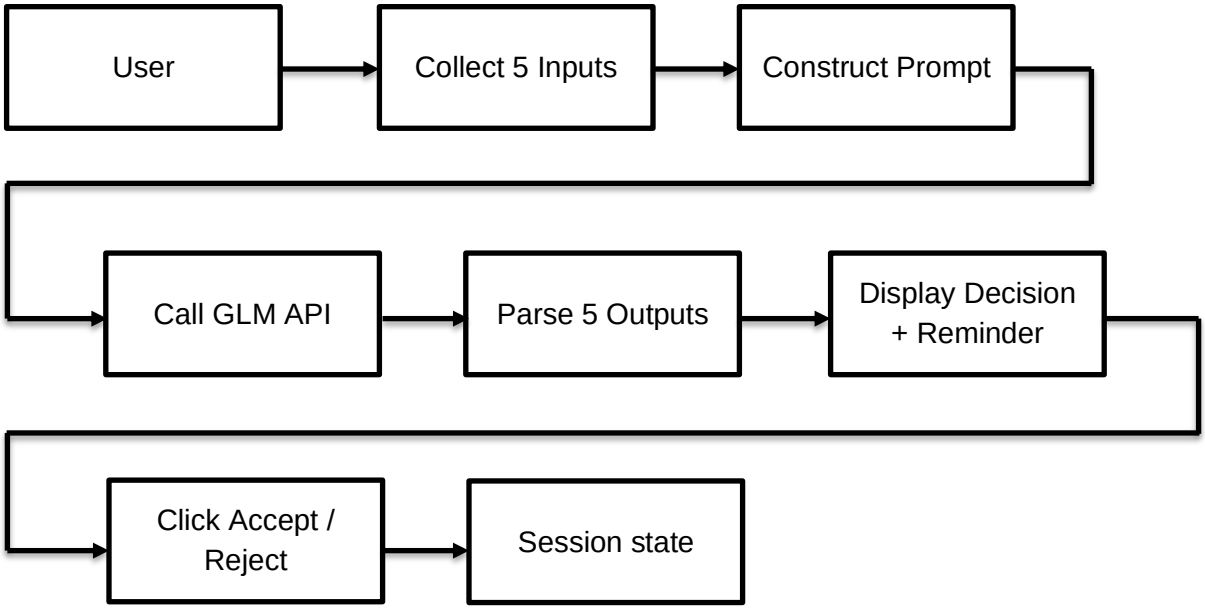
3. Call GLM API

4. Parse 5 Outputs

5. Display Decision + Reminder

6. Track Accept/Reject

c. Data Store: Session state



2.3.2. **Normalized Database Schema**

a. Not applicable – No database in MVP

© UMHackathon 2026

Email: umhackathon@um.edu.my

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product (MVP):

#	Features	Description
1	Monthly Spending Template	User enters spending categories (Food, Transport, Entertainment, etc.)
2	Monthly Allowance Input	User enters main monthly income
3	Passive Income Input	User enters additional income sources
4	Investment Input	User enters investment details
5	Future Goal Input	User enters goal + timeline
6	GLM API Call	System calls Z.AI GLM with all 5 inputs
7	5-Output Display	Shows Recommendation, Reasoning, Expected Impact, Confidence, Reminder
8	Decision Tracking	Accept/Reject buttons
9	Without GLM Comparison	Proves GLM critical path

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system.

Quality	Requirements	Implementation
Scalability	Handle concurrent demo users	Localhost only; future: async
Reliability	GLM success >95%	Retry logic; timeout 10s
Maintainability	Modular code	Separate glm_integration.py
Token Latency	GLM response <8s	Timeout 10s; loading spinner
Cost Efficiency	Token per session <1500	Fixed prompt; no history

© UMHackathon 2026

3.3. *Out of Scope / Future Enhancements*

Do identify the high-level features that the team planned for "BudgetBrain" but not part of this UMHackathon 2026.

a. *Database storage for user history*
b. *Automatic expense tracking (bank API)*
c. *SMS/Email reminders (MVP shows reminder on screen only)*
d. *Investment performance calculations*
e. *Mobile app*
f. *Multi-user collaboration*

4. *Monitor, Evaluation, Assumptions & Dependencies*

4.1. *Technical Evaluation:*

4.1.1. ***Grayscale Rollout & A/B Testing**: Not applicable for preliminary. Future: Test different reminder phrasing styles.*

4.1.2. ***Emergency Rollback (ER) & Golden Release**: If GLM returns >10% invalid responses (missing 5 sections), fallback to mock.*

4.1.3. ***Priority Matrix:***

Priority	Trigger	Action
P0	*GLM API unavailable*	*Show error; contact organizers*
P1	*Missing Reminder section*	*Retry with temperature=0.1*
P2	*Response time >10s*	*Timeout; suggest retry*
P3	*User rejects recommendation*	*Log feedback*

4.2. *Monitoring:*

4.2.1. *Show the planning for your monitoring plan of the system and how it will trigger if any damage is detected of the health of the system.*

- *Manual monitoring only (preliminary round)*
- *QA tests validate all 5 outputs present*
- *Response time logged to console*

© UMHackathon 2026

4.3. Assumptions: *State key operational & environmental conditions that you assumed to be valid while development of system and deployment.*

a. *Users know their typical monthly spending*
b. *Users know their passive income sources*
c. *Users can articulate a future goal*
d. *GLM API will be provided*
e. *Mock responses acceptable for development*

4.4. External Dependencies: *Do list all the external tools that are applied to your system to function.*

Tools	***Purpose***	***Risks***
Z Ai GLM API	*Core LLM reasoning engine that handles prompt inference, multi-step financial analysis, and generates 5 structured outputs (Recommendation, Reasoning, Expected Impact, Confidence, Reminder).*	*HIGH - API rate limits or downtime will break core functionality. Mitigation: mock GLM fallback, retry logic (3 attempts), timeout handling.*
Streamlit	*Python web framework for building UI, managing 5 input forms, session state, and displaying 5 outputs.*	*LOW - Version compatibility issues. Mitigation: pin version in requirements.txt.*
Requests	*HTTP client library for sending API requests to GLM endpoint.*	*LOW - Network connectivity issues. Mitigation: 10-second timeout, retry logic, error messages.*

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline:

Day	Activities	Deliverables
Day 1	Project kickoff, team roles assigned, problem statement finalized, PRD draft started	Team sync completed, PRD outline
Day 2	PRD first draft completed, SAD outline created, mock GLM structure designed	PRD draft v1, SAD outline
Day 3	PRD finalization, SAD first draft, basic Streamlit UI structure built	PRD v1.0, SAD draft, UI skeleton
Day 4	SAD completion, mock GLM implementation finished, UI input fields (5 inputs) completed	SAD v1.0, mock GLM working, UI prototype
Day 5	GLM prompt engineering (5-output format), UI connected to mock GLM, basic testing	Working prototype with mock GLM
Day 6	**MENTORSHIP SESSION** - Present current prototype to mentor, receive feedback, identify improvements. After session: implement mentor feedback, refine prompt engineering, improve UI based on suggestions	Mentor feedback documented, improved prototype v2
Day 7	Real GLM API integration (if available), 5-output display implementation, Accept/Reject tracking, apply mentor recommendations	Full app with real/mock GLM, mentor feedback incorporated
Day 8	Testing execution (happy path, edge cases, hallucination), QATD document drafted, pitch deck creation started	Test results, QATD draft, pitch deck outline
Day 9	QATD finalization, pitch deck completion, video script writing, video recording (10-minute demo)	QATD v1.0, pitch deck final, video recorded
Day 10	Final document review, bug fixes, submission checklist verification, team sign-off, SUBMIT to portal	SUBMITTED

5.2. Team Members Role:

Member	Role	Responsibilities
Member 1	GLM Integration Lead	Working API call + error handling
Member 2	Frontend Developer	Complete working Streamlit app
Member 3	QA & Testing Lead	QATD document + test cases
Member 4	Documentation Lead	PRD + SAD documents
Member 5	Pitch & Video Lead	Pitch deck + 10-min video

© UMHackathon 2026

5.3. Recommendations:

a. Reminder validation: Ensure GLM always outputs Reminder section (retry if missing)
b. Goal parsing: Allow natural language for goals (GLM handles this well)
c. Future caching: Cache repeated analyses for same user to reduce token costs

© UMHackathon 2026